

4일 차

이진 분류 모델링

이번 차시에서 주목할 키워드

- 이진 분류
- 시그모이드 함수
- 이진 교차 엔트로피
- 혼동 행렬
- 분류 모델 평가 지표

4일 차 학습 목표

- 이진 분류를 이해하고 시그모이드 함수가 확률을 만드는 원리를 습득합니다.
- 단순한 로지스틱 회귀 모델부터 은닉층이 포함된 분류 모델까지 직접 비교하며 만듭니다.
- HR 데이터를 활용한 실습으로 데이터 준비→모델 설계→학습→평가로 이어지는 분류 모델링 과정을 실습합니다.

분류 모델링은 데이터를 정해진 범주(Class) 가운데 하나로 분류하는 작업입니다. 범주가 두 개인 경우를 이진 분류(Binary Classification)라고 하며 고객 이탈 예측이나 질병 진단 등 실무에서 자주 활용되는 모델입니다. 범주가 세 개 이상이면 다중 분류(Multi-class Classification)라고 합니다. 이번 차시에서는 먼저 은닉층에 대해 비즈니스 관점에서 어떻게 이해해야 하는지 설명합니다. 다음으로 딥러닝을 활용한 이진 분류 모델을 중심으로 핵심 개념을 이해하고 직접 구현하며 실전 모델링 역량을 키우겠습니다.

은닉층에 대한 비즈니스 관점 이해

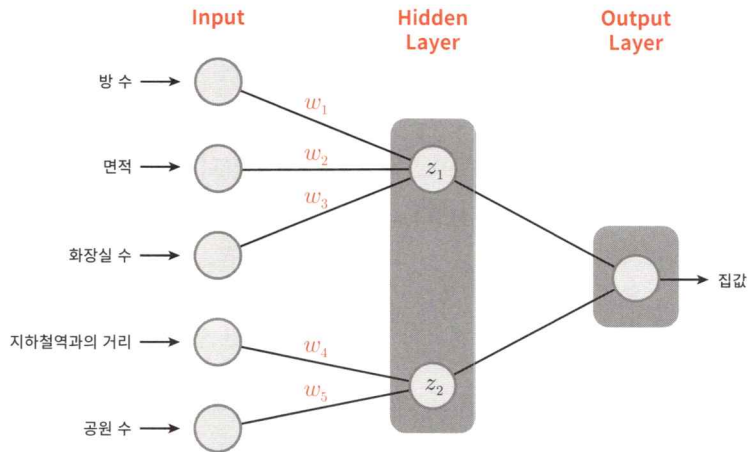
딥러닝 모델에서 은닉층은 입력 데이터를 변형하고 중요한 특징(feature)을 추출합니다. 하지만 은닉층 내부에서 정확히 어떤 일이 일어나는지 직접 관찰하기는 어렵습니다. 은닉층은 딥러닝 모델 구조에서 가장 중요한 부분으로 딥러닝의 꽃이라고 해도 과언이 아닙니다. 이번 절에서는 은닉층을 직관적으로 이해할 수 있는 특별한 모델 구조를 예로 들어 설명합니다.

은닉층에서 일어나는 일

은닉층의 역할은 기존 데이터를 변환하고 새로운 특징을 생성하는 일입니다. 생성(혹은 추출)된 특징들은 기존 데이터보다 더 의미 있고 문제 해결에 적합한 정보입니다. 3일 차에서 살펴본 ‘보스턴 집값 예측’ 데이터로 은닉층을 좀 더 이해해 보겠습니다.

[그림 4-1]과 같은 구조의 모델을 설계한다고 가정합니다. 노드 사이의 연결은 완전 연결(Fully Connected)이 아닌 부분 연결(Locally Connected) 구조입니다.

`nn.Linear` 함수로 층을 쌓으면 완전 연결 구조(3일 차 122쪽, [그림 3-11] 참조)가 되므로 [그림 4-1]과 같이 부분 연결 구조로 설계하려면 조금 특별한 함수들이 필요합니다. 이 모델을 코드로 구현하는 방법은 7일 차에서 다룹니다.



[그림 4-1] 집값 예측 모델(부분 연결 구조)



완전 연결, 부분 연결?

완전 연결(Fully Connected) 구조란 입력 정보가 층(Layer)의 각 노드와 모두 연결된 구조를 말합니다. 보통 모델 구조를 설계하면 완전 연결 구조가 됩니다. 완전히 연결된 층을 완전 연결 층(FC Layer)이라고 하며 뻥뻥하다는 의미에서 밀집층(Dense Layer)이라고도 합니다.

부분 연결(Locally Connected)은 모델에서 어떤 목적을 갖고 일부만 연결하도록 설계한 구조입니다. 부분 연결층(Locally Connected Layer)이라고도 합니다.

[그림 4-1]의 모델을 보면 입력 데이터는 방 수, 면적, 화장실 수, 지하철역과의 거리, 공원 수로 모두 5개입니다. 5개의 입력 데이터를 이용해 집값을 예측하려는 모델입니다. 1개의 은닉층은 2개의 노드로 구성되어 있습니다. 은닉층 각각의 노드는 입력값과 부분적으로 연결되어 있는 부분 연결 구조입니다.

첫 번째 노드에 연결된 입력값은 방 수, 면적, 화장실 수이고 입력값의 가중치는 w_1, w_2, w_3 입니다. 두 번째 노드에 연결된 입력값은 지하철역과의 거리, 공원 수이고 각각의 가중치는 w_4, w_5 입니다. 은닉층의 두 노드에서 계산되는 값은 z_1, z_2 라고 이름 붙였습니다. 집값은 최종 결과물이라고 하고 은닉층에서 계산된 값인 z_1, z_2 는 중간 결과물이라고도 합니다.

이 구조의 모델을 학습시키면 예측된 집값과 실제 집값의 차이(오차)를 최소화하는 방향으로 가중치가 업데이트됩니다. 예를 들어 에포크를 100으로 지정했다면 학습 데이터 전체를 100번 반복 학습하며 오차를 최소화하기 위해 가중치를 조정합니다.

생성한 모델을 사용할 때 은닉층에서는 어떤 일이 일어날까요? 은닉층에서 입력값을 받아 각각의 가중치를 곱해 계산된 값(중간 결과물)은 집값(최종 결과물)을 예측할 때 뭔가 특별한 의미를 제공하지 않을까요?

노드의 중간 결과물을 수식으로 표현하면 다음과 같습니다(다음 수식에서 $w_{1,0}$ 과 $w_{2,0}$ 은 바이어스(Bias) 파라미터입니다).

$$z_1 = w_1 \cdot \text{방 수} + w_2 \cdot \text{면적} + w_3 \cdot \text{화장실 수} + w_{1,0}$$

$$z_2 = w_4 \cdot \text{지하철역과의 거리} + w_5 \cdot \text{공원 수} + w_{2,0}$$

책을 더 읽기 전에 먼저 생각을 한 번 해봅시다. 은닉층에서 계산된 중간 결과물인 z_1 , z_2 의 값은 구체적으로 어떤 의미를 지닐까요? 수식의 입력 정보를 살펴보면 금방 떠올릴 수 있습니다.

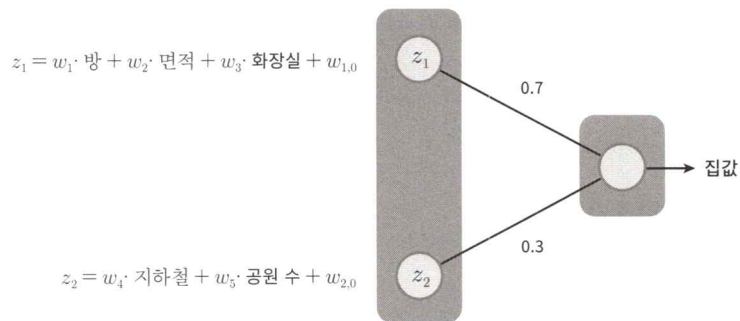
z_1 의 식을 보면 방 수, 면적, 화장실 수라는 입력값에 각각의 가중치를 곱해서 모두 더한 값입니다. 입력값들은 집을 구성하는 요소 중 내부 요인을 가리키는 값으로 생각할 수 있습니다. 따라서 z_1 을 집의 내부 요인 점수라고 표현하겠습니다. 이번엔 z_2 의 식을 봅시다. z_2 역시 지하철역과의 거리, 공원 수라는 입력값에 각각의 가중치를 곱하고 모두 더한 값입니다. z_1 과 비교할 때 z_2 는 집의 외부 요인으로 계산된 값이므로 이 값을 외부 요인 점수라고 부를 수 있겠죠.

이 과정을 다시 정리하면 다음과 같습니다.

1. 입력 정보로 집값을 최종적으로 예측하려고 합니다.
2. 학습을 통해 예측된 집값과 실제 집값의 차이(오차)를 최소화하도록 가중치가 조정됩니다.
3. 학습이 끝나면 오차를 최소화하도록 조정된 가중치가 저장됩니다. 학습의 목적은 바로 이 가중치를 찾는 일입니다.
4. 가중치로 구성된 모델을 사용할 때 중간에 있는 은닉층의 각 노드에서는 입력값에 가중치 $w_1, \dots, w_5$ 를 곱하고 더해 z_1, z_2 를 계산합니다.
5. 중간 결과물인 z_1, z_2 는 집값을 예측하는 데 필요한 뭔가 유익한 정보라고 추정할 수 있습니다.

여기서는 예제를 입력값의 부분 연결 구조가 눈에 띄도록 구성했으므로 z_1 과 z_2 가 집 내부 요인 점수와 집 외부 요인 점수라고 쉽게 이해할 수 있었습니다. 은닉층에서는 이처럼 입력 데이터를 조합해 더 의미 있는 표현을 만드는 과정이 일어납니

다. 이를 특징 표현(Feature Representation)이라고 하며 은닉층을 포함하는 딥러닝이 머신러닝보다 더 강력한 이유입니다.



[그림 4-2] 집값 예측 모델: 은닉층과 출력층

은닉층에서 계산된 내부 요인 점수와 외부 요인 점수는 출력층으로 연결되는데, 그때 가중치가 각각 0.7, 0.3이라면 집값에 대한 예측값은 다음과 같이 계산할 수 있습니다.

$$\text{집값(예측값)} = 0.7 \times \text{내부 요인 점수} + 0.3 \times \text{외부 요인 점수}$$

이 식의 은닉층과 출력층의 관계로부터 집값을 해석할 수 있습니다. 이는 모델로 집값을 예측할 때 내부 요인은 0.7, 외부 요인은 0.3의 중요도로 계산된다는 뜻입니다.

학습이 완료된 모델에 입력값을 넣으면 출력값, 즉 예측값이 나옵니다. 그러나 수많은 가중치와 은닉층에서 어떤 일이 벌어지는지 우리가 세세히 알기는 어렵습니다. 그래서 속을 들여다보거나 이해하기 어렵다는 의미로 딥러닝 모델을 블랙박스(Blackbox) 모델이라고 합니다.

딥러닝의 핵심은 “자동으로 유용한 특징을 학습하는 것”이며 이런 일이 바로 은닉층에서 일어납니다. 은닉층에서 정확히 어떤 일이 벌어지는지 알기 어렵습니다. 그러나 학습이 충분히 이루어졌다면 은닉층에서 계산된 값은 최종 출력을 만들기 위한 유익한 중간 결과물로 추정할 수 있습니다. 딥러닝 모델은 사람이 직접 가공하지 않아도 스스로 특징(중간 결과물)을 추출해 내는 능력이 있고, 우리는 그 과정을 신뢰하며 그 결과물을 활용할 수 있습니다. 그래서 필자는 강의할 때 종종 “이 부분은 믿음이 필요하다”라고 농담 삼아 말합니다. 내부 구조를 완전히 해석하기는 어렵지만, 학습된 결과를 신뢰할 수 있으며 충분히 활용할 수 있다는 의미입니다.

은닉층의 특징을 간단히 정리하면 다음과 같습니다.

- 입력 데이터를 가중치와 활성화 함수로 조합해 새로운 특징을 만듭니다.
- 생성된 특징은 원래 데이터보다 더 의미 있고 문제 해결에 적합한 정보입니다.
- 학습 과정에서는 손실 함수를 기준으로 오차를 줄이도록 가중치가 계속 조정됩니다.

이진 분류 개념 이해

함수의 기본 개념

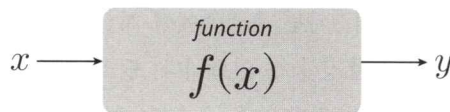
딥러닝 모델에서 활성화 함수(Activation Function)는 노드의 출력을 결정하는 중요한 역할을 수행합니다. 본격적으로 이진 분류 모델의 출력층에서 사용되는 활성화 함수인 시그모이드를 설명하기 전에 먼저 일반적인 함수는 어떤 기능을 수행하는지 알아 둘 필요가 있습니다.

수학에서 함수는 입력값을 변환해 출력값을 생성하는 규칙이라고 정의할 수 있습니다. 그리고 함수에는 다음과 같은 특징들이 있습니다.

- 입력값을 넣으면 특정 규칙에 따라 출력값이 결정됨
- 입력값을 x , 출력값을 y 로 표현함
- 즉, x 를 y 로 변환(Transformation)하는 역할

함수는 수학뿐만 아니라 프로그래밍이나 머신러닝, 딥러닝에서도 매우 중요한 개념입니다. 예를 들어 집의 크기에 따라 집값을 예측하는 모델을 만든다면 이 모델은 집의 크기를 입력으로 받아 집값을 출력하는 함수의 역할을 합니다. 결국 모델은 하나의 함수라고 말할 수 있습니다.

[그림 4-3]에서는 함수를 간단한 다이어그램으로 표현했습니다.



[그림 4-3] 함수 그림

함수는 수식으로 표현할 수 있으며 보통 다음과 같이 표기합니다.

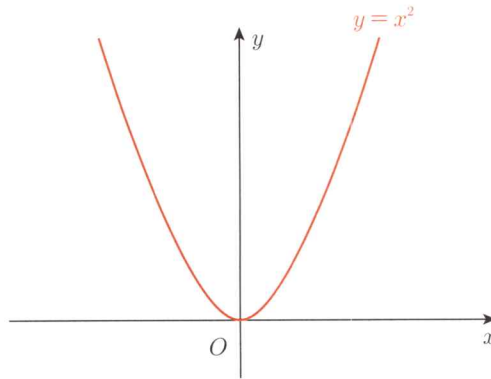
$$y = f(x)$$

함수 $f(x)$ 는 입력 x 를 받아 특정 연산을 수행한 다음, 출력 y 를 반환합니다.

여러분에게 익숙한 함수를 예로 들겠습니다. 다음 함수는 포물선을 그리는 2차 함수입니다.

$$f(x) = x^2$$

예를 들어 $x = [1, 2, 3, 4, 5]$ 이면 2차 함수는 각각의 값을 제공해 $y = [1, 4, 9, 16, 25]$ 라는 결과값을 생성합니다. 그래프로는 [그림 4-4]와 같으며, 이를 보면 입력값에 따라 출력값이 어떻게 변하는지 직관적으로 이해할 수 있습니다.

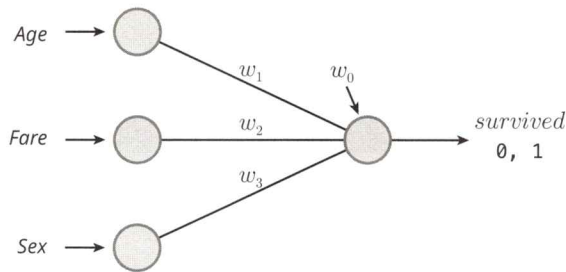


[그림 4-4] 2차 함수 그래프

이진 분류의 출력층

이진 분류 모델은 입력 데이터를 두 개의 범주 중 하나로 분류하는 작업을 수행합니다. 예를 들어 이번 차시에서 실습 예제로 다루게 될 타이타닉호 생존자 예측 모델에서 승객이 생존(Survived=1)했는지, 사망(Survived=0)했는지를 예측하는 작업이 대표적인 이진 분류 문제입니다. 타이타닉호 생존자 예측 모델에서 승객을 두 경우 중 하나로 분류하기 위해 출력층에서 사용하는 활성화 함수를 살펴보면서 이진 분류의 변환 과정을 설명하겠습니다.

[그림 4-5]에서는 타이타닉호 생존자 예측 모델을 간단한 다이어그램으로 표현했습니다.



[그림 4-5] 타이타닉호 생존자 예측 모델의 기본 구조

[그림 4-5]를 보면 타이타닉호 탑승객들의 생존 여부를 예측하기 위해 3개의 입력변수로 나이(Age), 운임(Fare), 성별(Sex)이 주어졌습니다. 이 모델 구조를 식으로 적으면 다음과 같습니다.

$$f(x) = w_1 \cdot \text{Age} + w_2 \cdot \text{Fare} + w_3 \cdot \text{Sex} + w_0$$

입력변수는 단순한 선형 조합을 거쳐 하나의 숫자로 계산됩니다. 그런데 출력값이 나타내는 의미와 원하는 결과값에는 차이가 있습니다. 모델이 예측하려는 것은 누가 사망했고(0) 누가 생존했는지(1)입니다. 즉, 결과값은 0 또는 1이어야 합니다. 그러나 수식의 결과인 $f(x)$ 의 값은 0과 1이 아닌 매우 크거나 작은 값일 수 있습니다. 따라서 이 출력값을 0과 1 사이의 확률로 변환하는 과정이 필요합니다. 그리고 얻은 확률이 0.5보다 크면 '1(생존)', 0.5보다 작으면 '0(사망)'으로 판단하는 간단한 규칙을 적용할 필요가 있습니다.

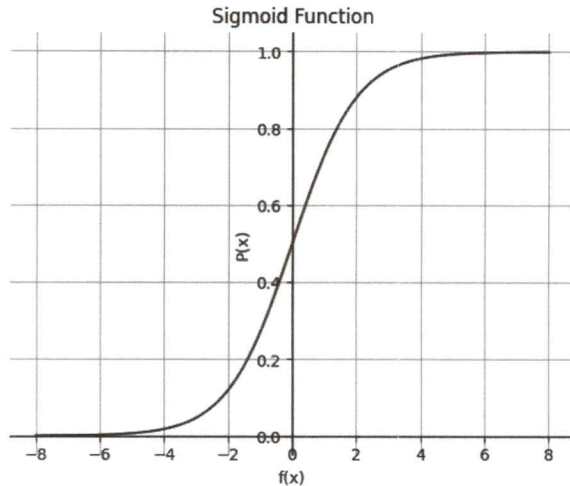
출력을 확률로 변환하는 활성화 함수: 시그모이드

이진 분류 모델에서는 모델의 최종 출력값을 0과 1 사이의 확률값으로 만들기 위해 시그모이드(Sigmoid) 활성화 함수를 사용합니다. 시그모이드 함수는 다른 말로 로지스틱(Logistic) 함수라고도 합니다. $f(x)$ 를 시그모이드 함수에 넣으면 확률값 $P(x)$ 으로 변환할 수 있습니다.

$$P(x) = \frac{1}{1 + e^{-f(x)}}$$

이 식의 그래프는 [그림 4-6]과 같습니다. 그래프에서 x 축은 입력축으로서 $f(x)$ 입니다. 또한 y 축은 출력축으로 $P(x)$ 로 표현되어 있습니다. 시그모이드 함수는 $-\infty \sim \infty$ 사이의 어떤 값을 입력으로 받아 0~1 사이의 확률값으로 출력하는 함수입니다.

로, 입력값이 클수록(∞ 에 가까울수록) 출력값은 1에 가까워지고 입력값이 작을수록($-\infty$ 에 가까울수록) 출력값은 0에 가까워집니다. 입력값이 0이면 출력값은 0.5입니다.



[그림 4-6] 시그모이드 그래프

임의의 탑승객 정보를 제공하고 이 수식으로 탑승객의 생존 여부를 실제로 예측해 봅시다. 탑승객의 정보는 Age = 25, Fare = 50, Sex = 1(남성)입니다(가중치 역시 임의로 정했습니다). 먼저 $f(x)$ 를 계산한 다음, 이를 시그모이드에 입력하면 다음과 같은 결과를 얻을 수 있습니다.

$$f(x) = 0.02 \cdot \text{Age} + 0.05 \cdot \text{Fare} + (-0.5) \cdot \text{Sex} + 0.1 = 0.02 \cdot 25 + 0.05 \cdot 50 - 0.5 \cdot 1 = 0.1$$

$$P(x) = \frac{1}{1 + e^{-f(x)}} = \frac{1}{1 + e^{-0.1}} \approx 0.525$$

결과로 얻은 확률은 0.525입니다. 1이 생존이므로 이 확률값을 생존 확률이라고 할 수 있습니다. 0.525는 생존율이 높은 것은 아닙니다만, 이 확률값을 0.5 기준으로 만들림하면 1이 되므로 '생존'이라고 예측할 수 있습니다.

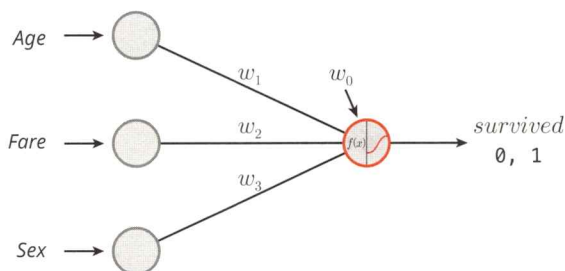
이와 같이 시그모이드 함수로 출력층의 노드에서 계산된 값(Logit, 로짓)을 예측 확률값으로 변환할 수 있습니다.



로짓(Logit)이란?

분류 모델은 출력층에 시그모이드 함수처럼 확률로 변환하는 함수가 붙습니다. 로짓은 시그모이드 함수에 들어가기 전, 즉 확률로 변환되기 전에 있던 원래의 입력값입니다. 예를 들어 모델이 어떤 입력을 받아 계산한 결과가 2.3이라면 이 값이 로짓이며 아직 확률값은 아닙니다. 시그모이드 함수는 이 로짓을 받아 0과 1 사이의 확률로 변환합니다. 로짓은 양수일수록 1에 가까운 확률을, 음수일수록 0에 가까운 확률을 만듭니다. 결국 모델은 로짓을 조정하면서 예측을 더 정확하게 하려고 학습을 진행합니다.

이진 분류 모델은 출력층의 결과를 확률값으로 변환하기 위해 시그모이드 함수를 사용합니다. 시그모이드 함수를 포함하는 딥러닝 모델을 표현할 때는 [그림 4-7]과 같이 그림니다. 즉, 출력층의 노드를 반으로 나누고 오른쪽에 시그모이드 그래프와 비슷한 'S'자 모양을 그려 넣습니다.



[그림 4-7] 시그모이드 함수가 포함된 이진 분류 모델의 기본 구조

이제 이진 분류 모델에서 출력층을 어떻게 구성하고 시그모이드 활성화 함수는 어떤 역할을 하는지 이해했습니다. 이어서 손실 함수(Loss Function)를 살펴보면서 모델을 어떻게 학습시키는지도 알아보겠습니다.

이진 분류 모델의 손실 함수

이진 분류 모델에서는 모델이 얼마나 잘 예측했는지 오차를 계산하기 위해 회귀 모델과는 다른 손실 함수(Loss Function)를 사용합니다. 회귀 모델에서는 평균 제곱 오차(Mean Squared Error, MSE)라는 손실 함수를 이용해 모델이 예측한 값과 목표 값 사이의 차이를 측정했습니다. 그리고 학습 과정에서는 이 손실값을 최소화하는 방향으로 모델의 파라미터를 업데이트했습니다. 이진 분류 모델에서는 어떻게 오차를 계산하는지 다음과 같은 예로 설명합니다.

[표 4-1] 이진 분류 모델의 목표값과 예측값 데이터 예제

y	$\hat{y}$
1	0.9
0	0.3
0	0.4
1	0.7
0	0.5
0	0.7
1	0.5

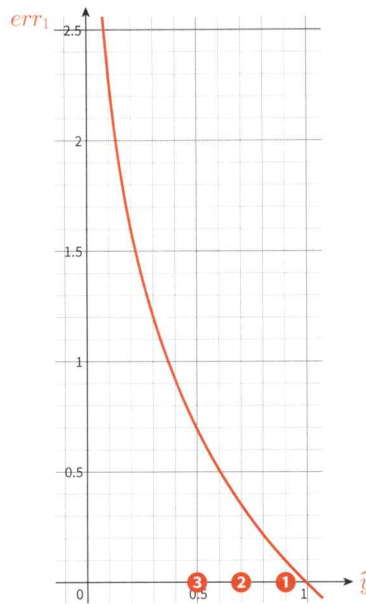
[표 4-1]을 보면 목표값(y)은 0과 1로 되어 있습니다. 예측값($\hat{y}$)은 0~1 사이의 확률 값입니다. 이 두 값 간의 차이, 즉 오차는 어떻게 계산하면 좋을까요? 목표값(y)이 1이라면 예측값($\hat{y}$)은 1에 가까울수록 오차가 작아지고 0에 가까울수록 오차가 커져야겠지요? 반대로 목표값(y)이 0이라면 예측값($\hat{y}$)은 0에 가까울수록 오차가 작아지고 1에 가까울수록 오차가 커지도록 계산되어야 합니다. 이런 오차 계산이 가능하도록 해주는 함수가 바로 교차 엔트로피(Cross Entropy, CE)입니다. 교차 엔트로피는 조금 어려울 수 있지만, 중요한 개념이므로 여러분이 꼭 알아야 합니다.

$y=1$ 일 때의 오차 계산

이진 분류 모델의 오차를 계산하는 손실 함수를 이진 교차 엔트로피(Binary Cross Entropy)라고 합니다. 이름이 좀 길지요? 이 손실 함수는 교차 엔트로피 식을 이진 분류에 맞게 조금 수정한 식입니다. 이 손실 함수를 이용해 오차를 계산합니다.

[그림 4-8] 왼쪽 그림에는 목표값과 예측값을 기록한 [표 4-1]을, 가운데 그림에는 목표값이 $y = 1$ 일 때 계산한 오차를 보여 줍니다. 예측값이 1에 가까울수록 오차는 줄어들고(0에 가까워지고), 예측값이 0에 가까울수록 오차는 ∞ 에 가까워지도록 계산합니다. 이렇게 계산하는 이유는 모델이 예측값을 1에 맞추도록 학습하기 위함입니다. [그림 4-8]의 오른쪽 그래프는 어떤 함수를 나타냅니다. 여기서는 어떤 함수라고 표현했지만, 곧 이 함수의 정체를 밝히겠습니다. 이 함수로 x 축(여기서는 $\hat{y}$ 의 값을 y 축(여기서는 오차 err_1)으로 변환합니다.

y	$\hat{y}$		err_1
1	0.9	① →	0.11
0	0.3		
0	0.4		
1	0.7	② →	0.36
0	0.5		
0	0.7		
1	0.5	③ →	0.69



[그림 4-8] y 가 1일 때 오차 계산 방식

[그림 4-8]에서 예측값($\hat{y}$)이 0.9(①)면 오차는 0.11로 계산되고, 예측값이 0.5(③)면 오차는 0.69로 계산됩니다. 또한 그래프를 보니 예측값이 0에 가까울수록 오차는 무한대에 가까워집니다. 이 어떤 함수를 이용하면 예측값이 1에 가까울수록 오차는 0에 가깝고, 예측값이 0에 가까울수록 오차는 굉장히 커지므로 원하는 오차 계산을 할 수 있습니다.

정리하면 $y = 1$ 인 경우에는

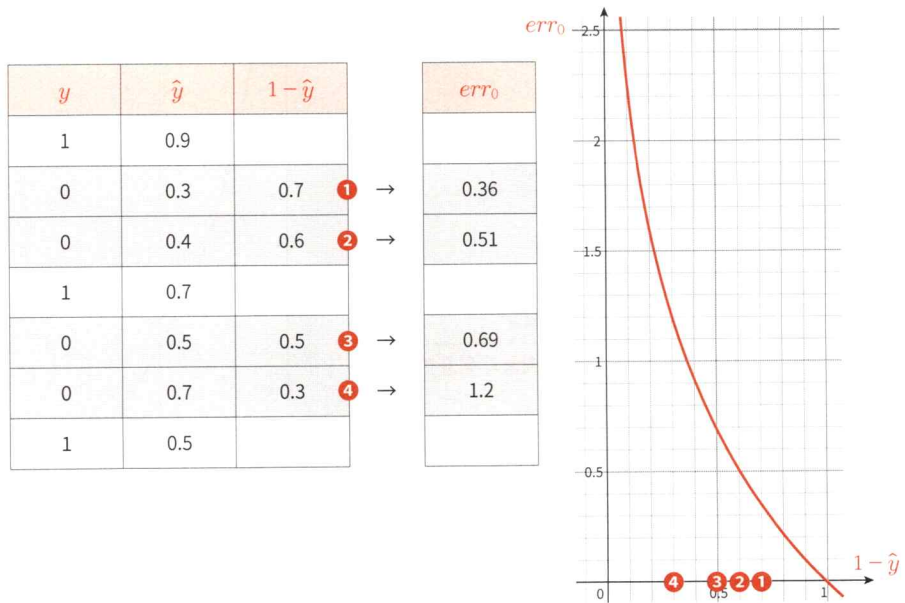
- $\hat{y}$ 이 1에 가까울수록 → 오차는 0에 가까워지도록
- $\hat{y}$ 이 0에 가까울수록 → 오차는 ∞ 에 가까워지도록

교차 엔트로피를 계산합니다.

$y=0$ 일 때의 오차 계산

목표값이 0인 경우 오차는 예측값이 0에 가까울수록 오차는 줄어들어야 하고, 예측값이 1에 가까울수록 오차는 무한대에 가까워져야 합니다. 그런데 이렇게 계산할 때 $y = 1$ 인 경우와 동일한 함수를 사용하려면 어떻게 해야 할까요? 굉장히 쉬운 방법이 있습니다. 1에서 예측값을 빼면 됩니다. [그림 4-9]의 왼쪽 표처럼 (1 - 예측값)

을 계산한 결과는 1에 가까울수록 오차가 작아집니다. $y=1$ 인 경우와 동일하게 계산됩니다.



[그림 4-9] y 가 0일 때 오차 계산 방식

목표값 0을 맞추기 위해 예측한 값이 0.3(❶)이라면 $1-0.3$ 을 계산해 0.7을 만든 다음, 이 함수에 넣으면 오차가 바르게 계산됩니다.

정리하면 $y=0$ 인 경우에는

- $\hat{y}$ 이 0에 가까울수록 → 오차는 0에 가까워지도록
- $\hat{y}$ 이 1에 가까울수록 → 오차는 ∞ 에 가까워지도록

교차 엔트로피를 계산합니다.

이진 분류에서 사용하는 손실 함수: 이진 교차 엔트로피

이 두 가지의 경우($y=0$ 과 $y=1$)를 하나의 식으로 묶어 계산하는 손실 함수가 바로 이진 교차 엔트로피(Binary Cross Entropy)입니다. y 가 0인 경우의 오차와 1인 경우의 오차를 모두 계산한 다음, 평균을 내어 전체 오차를 계산합니다.

y	$\hat{y}$
1	0.9
0	0.3
0	0.4
1	0.7
0	0.5
0	0.7
1	0.5

→

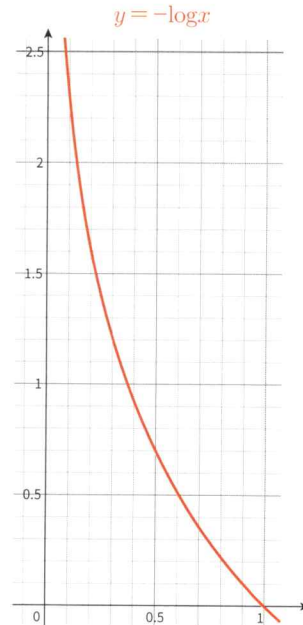
→

→

→

err_1	err_0
0.11	
	0.36
	0.51
0.36	
	0.69
	1.2
0.69	

이 오차들의 평균 계산



[그림 4-10] 이진 분류 모델의 오차 계산 방식

그리고 이를 계산하는 ‘어떤 함수’는 바로 $-\log$ 입니다. [그림 4-10]의 오른쪽 그래프가 $-\log$ 그래프입니다.

이진 교차 엔트로피의 함수식은 다음과 같습니다.

$$\text{Binary Cross Entropy} = -\frac{1}{n} \sum (y \cdot \log \hat{y} + (1 - y) \cdot \log (1 - \hat{y}))$$

조금 복잡해 보이지만, 방금 설명한 두 경우의 오차를 각각 계산해서 평균을 내는 식입니다. $\sum ()$ 의 첫 번째 항인 $y \cdot \log \hat{y}$ 는 $y=1$ 일 때 계산되고, 두 번째 항인 $(1-y) \cdot \log(1-\hat{y})$ 은 $y=0$ 일 때 계산됩니다. 그리고 $\frac{1}{n} \sum ()$ 로 모두 더하고 개수(n)로 나누어 평균을 구합니다. 식에 마이너스가 붙는 이유는 오차를 계산하는 식이 $-\log$ 이므로 마이너스를 앞으로 댄 것입니다. 참고로 수학식을 볼 때 $\frac{1}{n} \sum ()$ 가 있으면 “평균을 계산하는구나”라고 생각하면 됩니다.

지금까지의 내용을 다시 정리하면 다음과 같습니다.

- 이진 분류 모델에서는 예측값(확률값)과 목표값(0 또는 1) 사이의 차이를 측정하는 손실 함수가 필요합니다.
- $y=1$ 일 때, 예측값이 1에 가까울수록 오차가 작아지고 0에 가까울수록 오차가 커집니다.

- $y = 0$ 일 때, 예측값이 0에 가까울수록 오차가 작아지고 1에 가까울수록 오차가 커집니다.

복잡해 보였던 손실 함수를 하나씩 뜯어서 살펴보니 생각보다 어렵지 않지요? 이진 분류를 위한 중요 개념들을 익혔으니, 이제 본격적으로 이진 분류 모델링을 코드로 구현하겠습니다.

이진 분류 모델링 실습 1: 로지스틱 회귀

이제 새 코랩 파일을 생성하고 이진 분류(Binary Classification) 모델링 코드를 작성합니다. 3일 차에서 작성한 코드 파일을 이용해도 좋지만, 코드는 반복적으로 작성해야 손에 익습니다. 새 코랩 파일의 이름은 '4일차_이진분류모델실습.ipynb'로 합니다.

코드는 환경 준비, 데이터 전처리, 모델링 단계로 진행합니다.

환경 준비

환경 준비에서는 라이브러리 로딩, 디바이스 준비, 3일 차에서 설명했던 사용자 정의 함수들(make_DataSet, train, evaluate, dl_learning_curve)을 다시 생성한 다음, 데이터를 로딩합니다.

라이브러리 로딩과 디바이스 준비

3일 차 회귀 모델링에서 불러왔던 라이브러리와 동일합니다. 다음과 같이 라이브러리와 함수들을 불러옵니다.

CODE

```
# 기본 라이브러리
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# 전처리와 평가를 위한 sklearn 라이브러리
from sklearn.model_selection import train_test_split
from sklearn.metrics import *
from sklearn.preprocessing import MinMaxScaler

# 파이토치 라이브러리
import torch
```

```
from torch import nn
from torch.utils.data import DataLoader, TensorDataset
from torch.optim import Adam
```

코드에는 기본 라이브러리, 데이터 전처리와 평가를 위한 `sklearn` 라이브러리, 그리고 파이토치 라이브러리로 구분해 주석을 붙였습니다.

이어서 디바이스를 준비합니다.

```
CODE
device = "cuda" if torch.cuda.is_available() else "cpu"
```

새 코랩 파일에는 아직 GPU를 설정하지 않았으므로 변수 `device`에는 `cpu`가 저장됩니다. 다음으로 3일 차에서 만들었던 사용자 정의 함수들을 생성합니다.

사용자 정의 함수 생성

4가지 함수(`make_DataSet`, `train`, `evaluate`, `dl_learning_curve`)를 하나씩 생성합니다. 각각의 함수에는 딥러닝 학습에 꼭 필요한 절차들이 담겨 있습니다. 먼저 `make_DataSet` 함수입니다.

```
CODE
def make_DataSet(x_train, x_val, y_train, y_val, batch_size = 32) :
    # 데이터 텐서로 변환
    x_train_tensor = torch.tensor(x_train, dtype=torch.float32)
    y_train_tensor = torch.tensor(y_train, dtype=torch.float32).view(-1, 1)
    x_val_tensor = torch.tensor(x_val, dtype=torch.float32)
    y_val_tensor = torch.tensor(y_val, dtype=torch.float32).view(-1, 1)

    # TensorDataset 생성: 텐서 데이터 세트로 합치기
    train_dataset = TensorDataset(x_train_tensor, y_train_tensor)

    # DataLoader 생성
    train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle = True)
    return train_loader, x_val_tensor, y_val_tensor
```

`make_DataSet` 함수는 데이터 세트(`x_train`, `x_val`, `y_train`, `y_val`)와 배치 크기(`batch_size`)를 입력으로 받아 학습 데이터 로더(`train_loader`)와 검증용 텐서 데이터 세트(`x_val_tensor`, `y_val_tensor`)를 만듭니다. 데이터를 처리하는 순서는 먼저 입력으로 받은 학습 및 검증 데이터 세트를 텐서로 변환합니다. 그런 다음 학습

데이터 텐서는 텐서 데이터 세트로 만들어 데이터 로더를 생성하는 데 사용합니다.

다음으로 모델을 학습시키는 `train` 함수입니다.

CODE

```
def train(dataloader, model, loss_fn, optimizer, device):
    size = len(dataloader.dataset)  # 전체 데이터 세트의 크기
    num_batches = len(dataloader)  # 배치 크기
    tr_loss = 0

    model.train()  # 학습 모드로 설정
    for x, y in dataloader:  # 배치 단위로 로딩
        x, y = x.to(device), y.to(device)  # 디바이스 지정

        # Feed Forward(오차 순전파)
        pred = model(x)
        loss = loss_fn(pred, y)
        tr_loss += loss

        # Backpropagation(오차 역전파)
        loss.backward()  # 역전파를 통해 각 파라미터에 대한 오차의 기울기 계산
        optimizer.step()  # 옵티마이저가 모델의 파라미터를 업데이트
        optimizer.zero_grad()  # 옵티마이저의 기울기값 초기화
    tr_loss /= num_batches  # 모든 배치의 오차 평균
    return tr_loss.item()
```

`train` 함수는 데이터 로더(`dataloader`), 모델(`model`), 손실 함수(`loss_fn`), 옵티마이저(`optimizer`), 디바이스(`device`)를 입력으로 받아 학습시킨 다음, 학습 오차를 반환합니다. 특히 `train` 함수는 모델 학습의 핵심 개념인 오차의 순전파와 역전파 과정을 코드로 구현하고 있습니다. 또한 데이터 세트에서 배치 단위로 로딩해 학습시키는 부분 역시 주의해서 살펴보길 바랍니다.

이번에는 모델을 검증, 평가하는 `evaluate` 함수를 생성합니다.

CODE

```
def evaluate(x_val_tensor, y_val_tensor, model, loss_fn, device):
    model.eval()  # 모델을 평가 모드로 설정
    with torch.no_grad():  # 평가 과정에서 기울기를 계산하지 않도록 설정
        x, y = x_val_tensor.to(device), y_val_tensor.to(device)
        pred = model(x)
        eval_loss = loss_fn(pred, y).item()  # 예측값 pred와 목표값 y 사이의 오차 계산
    return eval_loss, pred
```

`evaluate` 함수는 학습이 완료된 모델을 검증 평가하기 위해 검증 데이터 세트(`x_`

val_tensor, y_val_tensor)와 학습된 모델(model), 손실 함수(loss_fn), 옵티마이저(optimizer), 디바이스(device)를 입력으로 받습니다. 함수는 검증 오차(eval_loss)와 검증 데이터 세트로 예측한 결과(pred)를 반환합니다. 검증은 학습 절차에 비해 간단합니다. 모델에 검증 데이터의 입력값(x_val_tensor)을 넣고 예측값(pred)을 출력한 다음, 목표값(y_val_tensor)과 예측값 사이의 오차를 계산합니다.

마지막으로 학습 곡선을 그리는 dl_learning_curve 함수를 생성합니다.

CODE

```
def dl_learning_curve(tr_loss_list, val_loss_list):
    epochs = list(range(1, len(tr_loss_list)+1))          # 에포크 수 계산
    plt.plot(epochs, tr_loss_list, label='train_err', marker = '.') # 학습 오차 그래프
    plt.plot(epochs, val_loss_list, label='val_err', marker = '.') # 검증 오차 그래프
    plt.ylabel('Loss')
    plt.xlabel('Epoch')
    plt.legend()
    plt.grid()
    plt.show()
```

dl_learning_curve 함수는 에포크 수만큼 학습을 반복할 때마다 학습과 검증 오차를 저장한 리스트를 이용해 그래프를 그리는 함수입니다.

이진 분류 모델에 필요한 4개의 함수를 모두 생성했습니다. 계속해서 데이터를 불러옵니다.

데이터 소개 및 로딩

이진 분류 모델링을 설명하기 위해 타이타닉호 탑승객 데이터를 사용합니다. 타이타닉호 탑승객 데이터 변수는 다음과 같습니다.

- PassengerId: 승객ID
- Survived: 탑승객의 생존 여부. 0: 사망, 1: 생존
- Pclass: 객실 등급. 1: 1등급, 2: 2등급, 3: 3등급
- Name: 승객 이름
- Sex: 성별. male, female
- Age: 나이. 0.42~80(개월 수를 년으로 환산한 값)
- Fare: 운임(단위: 달러)
- Embarked: 탑승지. S: Southampton, C: Cherbourg, Q: Queenstown

이제 데이터를 불러옵니다.

CODE

```
path = "https://bit.ly/titanic_simple_csv"
data = pd.read_csv(path)
data.head()
```

OUTPUT

	PassengerId	Survived	Pclass	Name	Sex	Age	Fare	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	7.2500	Southampton
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	71.2833	Cherbourg
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	7.9250	Southampton
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	53.1000	Southampton
4	5	0	3	Allen, Mr. William Henry	male	35.0	8.0500	Southampton

데이터를 로딩한 다음, 상위 5개의 행을 출력하면 5명의 승객 정보를 확인할 수 있습니다.

환경 준비가 모두 끝났습니다. 이어서 데이터 전처리 단계로 넘어갑니다.

데이터 전처리

앞서 살펴본 [그림 4-7]의 모델 구조에 맞게 변수 3개(Age, Sex, Fare)로 Survived(생존 여부)를 예측하는 이진 분류 모델을 생성하려고 합니다. 데이터 전처리 단계에서는 x, y 분할, 가변수화, train, val 분할, 스케일링 그리고 데이터 로더 및 텐서 데이터 세트 준비순으로 코드를 작성합니다.

x, y 분할

목표값(target)과 입력값(feature)을 정하고 각각 x와 y에 저장합니다. 여기서 사용할 입력변수는 Sex, Age, Fare입니다.

CODE

```
target = 'Survived'
features = ['Sex', 'Age', 'Fare'] ①
x = data.loc[:, features]
y = data.loc[:, target]
```

① 입력으로 사용할 변수 3개를 리스트로 지정합니다.

가변수화

다음으로 가변수화를 수행합니다. 3개의 입력변수 중에서 성별(Sex)은 범주형이므로 가변수화가 필요합니다. 다음과 같이 코드를 작성합니다.

CODE

```
x = pd.get_dummies(x, columns = ['Sex'], drop_first = True)
x.head()
```

OUTPUT

	Age	Fare	Sex_male
0	22.0	7.2500	True
1	38.0	71.2833	False
2	26.0	7.9250	False
3	35.0	53.1000	False
4	35.0	8.0500	True

코드에서 한 가지 눈여겨볼 부분은 가변수화할 범주형 변수가 1개여도 `pd.get_dummies` 함수에서는 `columns`의 값을 리스트 형태(['Sex'])로 지정한다는 점입니다. `pd.get_dummies`는 범주형 변수가 1개 이상일 때 사용하는 함수이므로 항상 리스트 형식으로 작성해야 합니다.

`pd.get_dummies` 함수에서 `drop_first` 옵션을 `True`로 지정했으므로 가변수화의 결과를 보면 여자 승객 변수(`Sex_female`)는 없어지고 `Sex_male`만 남습니다. `Sex_male`이 `True`면 남자, `False`면 여자입니다. 가변수화할 때는 두 변수 중 하나를 제거하고 남은 하나의 변수로 두 정보(남자, 여자)를 표현할 수 있습니다. 참고로 값은 논리값인 `True`와 `False`지만, 모델링에서 `True`는 1, `False`는 0으로 처리됩니다.

학습과 검증 데이터 분할 및 스케일링

이번에는 학습 데이터와 검증 데이터로 분할하고 스케일링합니다.

CODE

```
# 데이터 분할
x_train, x_val, y_train, y_val = train_test_split(x, y, test_size=.3)

# 스케일링
scaler = MinMaxScaler()
x_train = scaler.fit_transform(x_train)
x_val = scaler.transform(x_val)
```

```
# make_DataSet 입력을 위해 y_train, y_val를 넘파이 배열로 변환
y_train = y_train.values
y_val = y_val.values
```

`train_test_split` 함수를 이용해 `x`, `y`를 학습 데이터(`x_train`, `y_train`)와 검증 데이터(`x_val`, `y_val`)로 분할합니다. 이때 학습 데이터와 검증 데이터의 비율은 7:3으로 지정했습니다(`test_size=.3`).

또한 입력 데이터(`x_train`, `x_val`)를 MinMax 방식으로 스케일링합니다. MinMax 방식으로 스케일링하면 모든 입력변수의 범위를 0~1로 맞춥니다. 스케일링에서 꼭 기억해야 할 것이 한 가지 더 있습니다. 스케일링의 기준은 학습 데이터라는 것입니다. 검증이나 테스트 데이터는 학습 데이터로 만들어진 스케일링 기준을 적용할 뿐입니다. 예를 들어 학습 데이터에서 탑승객의 나이가 10~80세의 범위에 있다고 가정할 때, 학습 데이터를 MinMax 방식으로 스케일링하면 10~80세의 범위가 0~1 사이의 범위로 압축 변환됩니다. 즉, 10세는 0, 80세는 1로 변환되는 것이죠. 그리고 10세에서 80세 사이의 나이는 0과 1 사이의 값이 됩니다. 검증과 테스트 데이터는 이 학습 데이터의 값을 기준값으로 해서 계산됩니다.

데이터 로더

기본적인 데이터 세트가 준비되었으므로 데이터 로더로 변환합니다. 학습 데이터는 데이터 로더를 이용해 배치 단위로 학습에 사용됩니다. 하지만 검증이나 테스트 데이터 세트는 배치 단위 처리가 꼭 필요하지 않습니다. 모델로 예측(추론)만 하면 되므로 텐서 데이터 세트를 그대로 이용해도 됩니다.

앞서 생성한 `make_DataSet` 함수를 사용해 학습용 데이터 로더와 검증용 텐서 데이터 세트를 만듭니다.

CODE

```
train_loader, x_val_ts, y_val_ts = make_DataSet(x_train, x_val, y_train, y_val, 32)

# 첫 번째 배치만 로딩해서 살펴보기
for x, y in train_loader:
    print(f"Shape of x [rows, columns]: {x.shape}")
    print(f"Shape of y: {y.shape} {y.dtype}")
    break
```

OUTPUT

```
Shape of x [rows, columns]: torch.Size([32, 3])
Shape of y: torch.Size([32, 1]) torch.float32
```

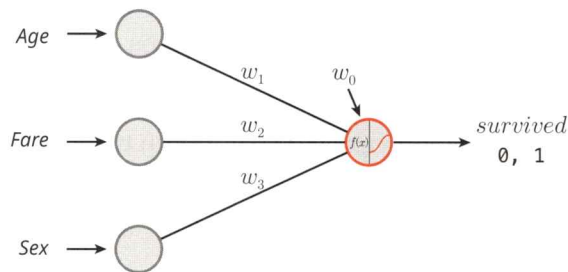

데이터 로더를 준비하고 첫 번째 배치의 구조(shape)를 출력했습니다. 배치 하나에 x 는 [32, 3](32행 3열), y 는 [32, 1](32행 1열)로 구성되어 있음을 알 수 있습니다.

이제 모델링을 위해 필요한 데이터 준비가 모두 끝났습니다. 다음 단계로 이진 분류 모델링을 직접 수행하겠습니다.

이진 분류 모델링

모델링 절차는 모델 선언(모델 구조 설계), 학습, 학습 곡선 확인, 모델 검증 및 평가 순서로 진행됩니다.

먼저 모델을 선언합니다. [그림 4-11]은 코드와 비교해서 볼 수 있게 앞서 소개한 다이어그램을 다시 가져온 것입니다.



[그림 4-11] 세 입력변수로 생존 여부를 예측하는 모델 구조

모델 구조 설계

모델은 [그림 4-11]과 같이 노드가 하나 있는 층(Layer)에서 3개의 입력을 받아 출력 값을 계산하는 구조입니다. 다음과 같이 코드를 작성합니다.

```
CODE
n_feature = x.shape[1]
# 모델 구조 설계
model = nn.Sequential( nn.Linear(n_feature, 1), ①
                       nn.Sigmoid()            ②
                       ).to(device)
```

① `nn.Linear(n_feature, 1)`: 3개의 입력 데이터를 1개의 출력 노드와 연결합니다.
 ② `nn.Sigmoid()`: 시그모이드 활성화 함수를 이용해 출력 결과를 0~1 사이의 확률값으로 변환합니다. 회귀 모델에는 없는 부분입니다.

간단한 이진 분류 모델의 구조입니다. 회귀 모델과 차이가 있다면 출력층 다음에

시그모이드 함수를 추가하는 부분입니다. 시그모이드를 왜 추가했는지는 앞에서 설명했습니다.

모델을 선언했다면 손실 함수와 옵티마이저도 준비합니다.

CODE

```
loss_fn = nn.BCELoss() ①
optimizer = Adam(model.parameters(), lr=0.01)
```

| ① 손실 함수로 BCELoss(이진 교차 엔트로피, Binary Cross Entropy Loss)를 사용합니다.

이진 분류 모델 구조는 회귀 모델을 설계할 때와 다른 점이 두 가지 있습니다. 이진 분류 모델은 출력층에 시그모이드 함수를 붙이고 손실 함수로 BCELoss를 사용한다는 점입니다(회귀 모델의 손실 함수는 MSELoss입니다). 그 외에는 모두 동일합니다.

학습

모델 구조를 선언하고 손실 함수와 옵티마이저를 준비했으니 이제 학습을 시킵니다. 학습 절차는 중요하므로 다시 한번 간략히 정리합니다.

1. 에포크 수만큼 학습 데이터를 반복 학습
2. 에포크마다 데이터 로더를 이용해 배치 단위로 데이터를 로딩
3. 배치마다 학습 과정(오차의 순전파와 역전파)을 진행하면서 가중치 업데이트

이 과정을 머릿속에서 그릴 수 있어야 합니다. 학습을 위한 코드를 작성합니다.

CODE

```
epochs = 50
tr_loss_list, val_loss_list = [], []
for t in range(epochs):
    tr_loss = train(train_loader, model, loss_fn, optimizer, device) ①
    val_loss, _ = evaluate(x_val_ts, y_val_ts, model, loss_fn, device) ②
    # 리스트에 loss 추가
    tr_loss_list.append(tr_loss)
    val_loss_list.append(val_loss)
    print(f"Epoch {t+1}, train loss : {tr_loss:4f}, val loss : {val_loss:4f}")
```

OUTPUT

```
Epoch 1, train loss : 0.720467, val loss : 0.678466
Epoch 2, train loss : 0.696399, val loss : 0.661170
Epoch 3, train loss : 0.673542, val loss : 0.647313
...
Epoch 48, train loss : 0.477550, val loss : 0.561683
```

Epoch 49, train loss : 0.485161, val loss : 0.562236
Epoch 50, train loss : 0.480898, val loss : 0.562290

- ① `tr_loss = train(train_loader, model, loss_fn, optimizer, device)`: 에포크 단위로 학습을 수행하고 학습 오차를 반환합니다.
- ② `val_loss, _ = evaluate(x_val_ts, y_val_ts, model, loss_fn, device)`: 생성된 모델로 예측을 수행하고 검증 오차를 반환합니다. `evaluate` 함수는 결괏값이 두 개인데, 첫 번째는 검증 오차이고 두 번째는 예측 결과를 담은 텐서입니다. 학습 단계에서는 검증 데이터의 예측 결과가 필요하지 않으므로 `_`로 처리합니다.

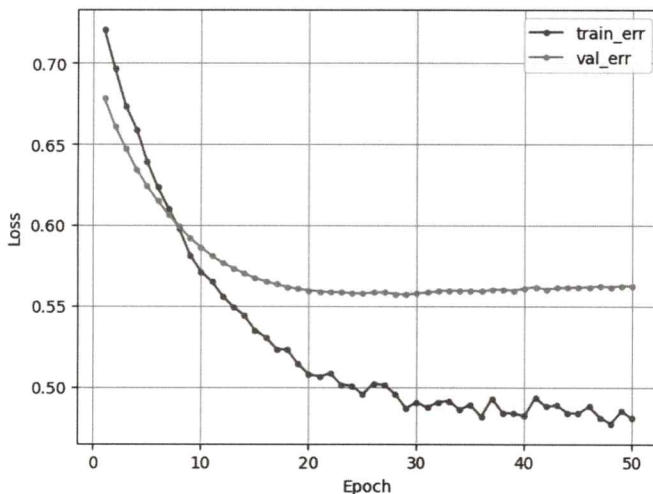
코드를 보면 에포크는 50으로 하고 학습 오차와 검증 오차의 값을 저장할 빈 리스트를 선언했습니다. 그리고 학습 데이터를 에포크 수만큼 반복 학습시키면서 학습 오차와 검증 오차 데이터를 리스트에 저장합니다.

학습 곡선

학습을 모두 마치면 학습이 적절히 진행되었는지 검토하기 위해 학습 곡선을 그려 봅니다. 다음과 같이 코드를 작성합니다

```
CODE
dl_learning_curve(tr_loss_list, val_loss_list)
```

실행하면 [그림 4-12]와 같은 학습 곡선이 출력됩니다.



[그림 4-12] 학습 곡선

[그림 4-12]의 학습 곡선을 보면 에포크 초기에 오차가 급격히 줄고 대략 에포크 10~20 사이부터 오차의 감소 정도가 완만해집니다. 또한 에포크 20 이후로 학습 오차 곡선이 들쭉날쭉하는데, 학습률을 조금 줄일 필요가 있어 보이지만 여기서는 이진 분류 모델링의 수행 방법을 익히는 데 초점을 맞추므로 특별히 조정하지는 않습니다.

검증 평가

학습을 완료했으면 모델의 성능을 검증 평가합니다. 검증 평가는 `evaluate` 함수를 이용해 예측 결과(확률값)를 얻은 다음, 예측 결과를 0.5 기준으로 잘라 0과 1로 만들고 평가합니다.

예측

`evaluate` 함수로 검증 데이터의 예측값(pred)을 구합니다.

CODE

```
_, pred = evaluate(x_val_ts, y_val_ts, model, loss_fn, device)
print(pred.numpy()[:5])
```

OUTPUT

```
[[0.6970004 ]
 [0.15660214]
 [0.20633022]
 [0.17745827]
 [0.7065605 ]]
```

5건의 예측 결과(pred)를 출력하면 값은 각각 0.697, 0.157, 0.206, 0.177, 0.707입니다(소수점 넷째 자리에서 반올림). 이 값은 출력층의 값을 시그모이드로 변환한 확률값입니다. 생존이 1이고 사망이 0이므로 이 값을 '생존에 대한 예측 확률', 줄여서 '생존율'이라고 할 수 있습니다. 코드에서 `evaluate` 함수의 첫 번째 출력값인 오차(손실값, BCELoss 값)는 여기서 필요하지 않으므로 `_` 처리합니다.

확률값을 잘라 0과 1로 변환

예측값으로 나온 확률값(0~1 사이의 값)을 목푫값인 0, 1과 비교하려면 확률값을 0 또는 1로 잘라 주어야 합니다. 보통은 0.5를 기준으로 소수점 첫째 자리에서 반올림하는데, `np.where` 함수를 이용하면 손쉽게 수행할 수 있습니다.

CODE

```
pred = np.where(pred.numpy() > .5, 1, 0) ①
print(pred[:5])
```

OUTPUT

```
[[1]
 [0]
 [0]
 [0]
 [1]]
```

① `np.where(pred.numpy() > .5, 1, 0)`은 0.5보다 크면 1, 작으면 0을 반환합니다. `np.where` 함수는 텐서를 인식하지 못하므로 텐서인 `pred`에 `numpy` 메서드를 사용해 넘파이 배열로 변환해야 합니다.

**확률을 0과 1로 나누는 기준 0.5**

딥러닝 이진 분류 모델에서는 시그모이드 함수의 결과로 0과 1 사이의 확률값이 출력됩니다. 이 확률은 “이 샘플이 클래스 1에 속할 가능성이 얼마나 되는가?”를 의미합니다. 하지만 모델은 확률값을 출력할 뿐 실제로 어떤 범주로 분류할지는 나누는 기준(cut-off value)에 따라 결정됩니다. 일반적으로 0.5보다 크면 분류 1, 작으면 분류 0으로 판단합니다. 즉, 확률이 절반 이상이면 ‘맞다’, 미만이면 ‘아니다’로 예측하는 셈입니다.

이 기준은 직관적이고 대부분 무난하게 작동하지만, 언제나 최선의 기준은 아닙니다. 예를 들어 질병 진단과 같이 예민한 문제는 0.5보다 더 낮은 값으로 기준을 설정할 수 있습니다. 질병 진단에서는 약간의 가능성 있는 양성 사례도 놓치지 않고 잡는 일에 초점을 맞추기 때문입니다. 반대로 스팸 필터처럼 잘못된 분류가 사용자에게 큰 불편을 주는 경우에는 기준을 더 높이는 전략을 쓸 수도 있습니다. 확실한 스팸 메일만 걸러내겠다는 의미지요. 따라서 나누는 기준값(cut-off value)은 문제의 종류, 평가 지표에 따라 조정이 가능하다는 점도 함께 기억하면 좋습니다.

성능 평가

예측값을 0과 1로 만들었으니 목פות값 0, 1과 비교해 분류 모델을 평가합니다.

분류 모델은 `sklearn`에서 제공하는 혼동 행렬(Confusion Matrix) 함수로 집계하고 이를 바탕으로 함수 `classification_report`를 이용해 평가합니다.

CODE

```
print(confusion_matrix(y_val_ts.numpy(), pred))
```

OUTPUT

```
[[113 19]
 [ 37 45]]
```

CODE

```
print(classification_report(y_val_ts.numpy(), pred))
```

OUTPUT

	precision	recall	f1-score	support
	0.0 1.0	0.75 0.70	0.86 0.55	0.80 0.62
				132 82
accuracy			0.74	214
macro avg	0.73	0.70	0.71	214
weighted avg	0.73	0.74	0.73	214

코드에서 텐서를 넘파이 배열로 각각 변환(y_val_ts.numpy)하는 이유는 sklearn 함수 역시 텐서를 인식하지 못하기 때문입니다.

평가 결과를 보면 정분류율(Accuracy)은 0.74(❶)입니다. 검증 데이터 전체에서 생존자와 사망자를 맞춘 비율이 0.74라는 의미입니다. 생존자(❷) 관점의 정밀도(Precision)는 0.7(❸)로 식 $\frac{45}{19+45}$ 로 계산된 값입니다. 생존자로 예측한 값(19 + 45) 가운데 맞춘 값(45)의 비율이 0.7이라는 뜻입니다. 또한 재현율(Recall)은 0.55(❹, $\frac{45}{37+45}$)로 실제 생존자를 맞춘 비율입니다. 이 두 값의 조화 평균인 F1 score는 0.62(❺, $\frac{2 \times 0.7 \times 0.68}{0.7 + 0.68}$)로서 정밀도와 재현율의 평균을 의미합니다. 각 지표에 대한 자세한 설명과 공식은 1일 차 46~51쪽을 참조하기 바랍니다.

그런데 여기서 얻은 성능 평가 결과가 좋은지 나쁜지는 상황에 따라 달라질 수 있습니다. 모델의 정확도나 정밀도, 재현율 등은 절대적인 기준이 아니라 상대적인 지표이기 때문에 어떤 상황에서는 0.74의 정확도가 훌륭하게 받아들여질 수 있지만, 다른 상황에서는 부족하다고 판단할 수도 있습니다. 따라서 모델의 성능을 판단하려면 비교 대상이 있어야 합니다. 비교 대상은 기존 모델이 될 수도 있고 단순히 만든 모델을 기준으로 삼을 수도 있습니다. 보통 비교 대상이 되는 기존 모델을 베이스라인 모델(Baseline Model)이라고 합니다. 딥러닝에서는 베이스라인 모델과 비교했을 때 지금 학습한 모델이 충분히 의미 있는 성능을 내는지를 평가합니다.

이진 분류 모델링 실습 2: 은닉층 추가

이진 분류 모델링 1에서는 3개의 입력값으로 예측 모델을 만들었는데, 이번 절에서는 타이타닉호 탑승객 데이터에 있는 변수를 모두 이용하고 은닉층도 추가해 모델

을 학습시키겠습니다.

이진 분류 모델링 실습 2에서는 파일을 따로 생성하지 않고 '4일차_이진분류모델 실습.ipynb' 파일을 그대로 이용해 데이터 전처리와 모델 구조 설계, 학습 및 검증 평가를 수행합니다. 코드는 마지막 빈 셀에서 이어 작성하면 됩니다.

데이터 전처리

라이브러리 불러오기 → 디바이스 설정 → 함수 생성 → 데이터 로딩순으로 진행되는 코드는 앞서 작성한 코드와 동일합니다. 데이터 전처리부터 조금씩 달라지는데, 차근차근 하나씩 코드를 작성하기 바랍니다.

CODE

```
# 모델링에서 불필요한 열 제거
data = data.drop(['PassengerId', 'Name'], axis = 1) ①

# x, y 분할
target = 'Survived'
x = data.drop(target, axis=1)
y = data.loc[:, target]

# 가변수화
x = pd.get_dummies(x, columns = ['Pclass', 'Sex', 'Embarked'], drop_first = True) ②

# train, val 분할
x_train, x_val, y_train, y_val = train_test_split(x, y, test_size=.3)

# 스케일러 선언
scaler = MinMaxScaler()
x_train = scaler.fit_transform(x_train)
x_val = scaler.transform(x_val)

# make_DataSet 입력을 위해 y_train, y_val을 넘파이 배열로 변환
y_train = y_train.values
y_val = y_val.values

train_loader, x_val_ts, y_val_ts = make_DataSet(x_train, x_val, y_train, y_val, 32)
```

- ① 모델을 만들 때는 보통 일련번호와 이름 등은 제외합니다.
- ② 타이타닉호 탑승객 데이터에서 범주형 feature는 Pclass, Sex, Embarked 세 가지입니다.
get_dummies 함수에 이 세 항목을 리스트로 추가하고 가변수화합니다.

모델링에서 예측과 직접적인 관련이 없거나 단순히 행 구분자 역할을 하는 변수는 제외합니다. 예를 들어 PassengerId는 단순한 일련번호일 뿐이며, Name 역시 이름

이라는 문자열 정보가 생존 여부와 직접적으로 관련이 있다고 보기 어렵습니다. 특히 이름에는 중복이나 희귀 단어, 호칭(Mr, Mrs 등)이 섞여 있어 별도로 정제하지 않으면 오히려 모델의 학습을 방해할 수 있습니다. 따라서 이런 열들은 사전에 제거해 모델이 의미 있는 데이터에만 집중하도록 돕는 것이 좋습니다.

데이터 전처리 코드의 흐름을 다시 간단히 정리하면 다음과 같습니다.

x, y 분할 → 가변수화 → train, val 분할 → 스케일링 → 데이터 로더 준비

여러 번 반복했으니 이제 전처리 흐름이 눈에 익었으리라 생각합니다.

모델링

모델링 코드는 모델 구조 설계, 학습, 학습 곡선 확인, 모델 검증 및 평가 순서로 진행됩니다. 이진 분류 모델링 1의 코드와 비교할 때 모델 구조에 은닉층을 추가하는 부분만 차이가 납니다.

모델 설계: 전체 입력변수+은닉층 추가

이진 분류 모델 2에서 모델을 설계할 때는 전체 입력변수를 이용하고 은닉층을 추가합니다.

CODE

```
n_feature = x.shape[1] ①
# 모델 구조 설계
model = nn.Sequential(nn.Linear(n_feature, 5), ②
                      nn.ReLU(), ③
                      nn.Linear(5, 1), ④
                      nn.Sigmoid() ⑤
                      ).to(device)
loss_fn = nn.BCELoss()
optimizer = Adam(model.parameters(), lr=0.01)
```

- ① x에는 이미 전체 입력변수가 저장되어 있습니다. shape[1] 속성으로 전체 입력변수의 개수를 변수 n_feature에 저장합니다.
- ② 은닉층에서는 입력값의 개수(n_features)만큼 정보를 받아 5개의 노드와 완전 연결(Fully Connected)해 연산합니다.
- ③ 활성화 함수 ReLU는 ②의 은닉층에서 계산한 값 5개 중 값이 0보다 작으면 0, 크면 그대로 다음 층으로 전달합니다.
- ④ 출력층에서는 ③의 결과를 받아 1개의 출력값을 계산합니다. 이 값을 로짓이라고 하며 시그

모이드(Sigmoid) 함수를 통해 확률로 변환합니다.

⑤ 활성화 함수 Sigmoid는 ④의 로짓을 확률값으로 변환합니다.

코드에서는 은닉층을 추가하고 은닉층 다음에 활성화 함수 ReLU를 포함하고 있습니다. 이처럼 모델을 설계할 때 은닉층과 은닉층의 활성화 함수를 추가하는 일은 회귀나 분류 모두 똑같습니다.

학습

이어서 학습 코드를 작성합니다. 학습 코드는 이전과 동일하므로 이진 분류 모델링 1의 내용을 그대로 복사해서 사용해도 좋습니다.

CODE

```
epochs = 50
tr_loss_list, val_loss_list = [], []
for t in range(epochs):
    tr_loss = train(train_loader, model, loss_fn, optimizer, device)
    val_loss, _ = evaluate(x_val_ts, y_val_ts, model, loss_fn, device)
    # 리스트에 loss 추가
    tr_loss_list.append(tr_loss)
    val_loss_list.append(val_loss)
    print(f"Epoch {t+1}, train loss : {tr_loss:4f}, val loss : {val_loss:4f}")
```

OUTPUT

```
Epoch 1, train loss : 0.668021, val loss : 0.654244
Epoch 2, train loss : 0.645052, val loss : 0.633402
Epoch 3, train loss : 0.625608, val loss : 0.606892
...
Epoch 48, train loss : 0.445190, val loss : 0.390280
Epoch 49, train loss : 0.445122, val loss : 0.392608
Epoch 50, train loss : 0.436174, val loss : 0.391094
```

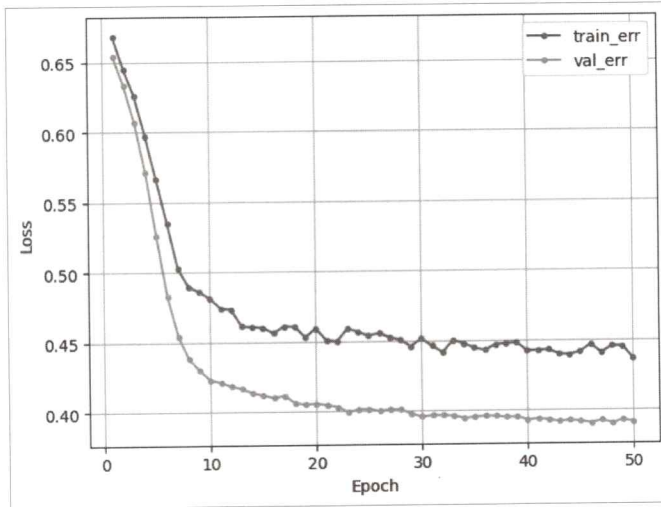
이전과 동일하게 50번을 반복 학습시킵니다. 에포크마다 학습 함수(train)와 평가 함수(evaluate)를 수행해 학습 오차와 검증 오차를 저장하고, 학습 곡선을 그릴 때 이 오차 데이터를 사용하기 위해 리스트에 추가합니다.

학습 곡선

학습이 적절히 수행되었는지 확인하기 위해 학습 곡선을 그립니다.

CODE

```
dl_learning_curve(tr_loss_list, val_loss_list)
```

OUTPUT

학습 곡선을 보면 오차가 급격히 줄다가 8~10 에포크 사이에서 꺾이면서 그래프의 기울기는 점차 완만해집니다.

검증 평가

은닉층을 포함한 모델을 학습했으므로 모델 성능을 평가하고 이전 모델과 비교하겠습니다.

학습된 모델과 검증 데이터 세트로 예측하고 성능을 평가하기 위해 다음과 같이 코드를 작성합니다.

CODE

```
# 예측
_, pred = evaluate(x_val_ts, y_val_ts, model, loss_fn, device)

# 예측 확률값을 0.5 기준으로 반올림
pred = np.where(pred.numpy() > .5, 1, 0)

# 분류 모델 평가 지표 계산
print(classification_report(y_val_ts.numpy(), pred))
```

	precision	recall	f1-score	support
0.0	0.81	0.94	0.87	127
1.0	0.89	0.68	0.77	87
accuracy			0.84	214
macro avg	0.85	0.81	0.82	214
weighted avg	0.84	0.84	0.83	214

입력변수 3개로 만든 이진 분류 모델링 1의 정분류율(Accuracy)은 0.74였는데, 이번에는 0.84로 성능이 약간 개선되었습니다. 변수를 추가하고 은닉층을 추가한 결과입니다. 데이터를 무작위로 나누고 학습할 때는 초기 가중치도 임의로 설정하기 때문에 이 책의 결과와 여러분이 얻는 결과는 다를 수 있습니다.

이진 분류 모델링 실습 3: 직원 이직 예측

이진 분류 모델링을 추가로 실습하겠습니다. 이번에는 비즈니스 상황을 이해하고 문제 해결을 위한 모델링 관점에서 실습할 수 있도록 설명합니다.

이번 실습의 주제는 ‘직원 이직 예측 모델링’입니다. 최근 기업들은 우수 인재 확보와 조직 안정성 유지를 위해 직원 이직을 예측하는 일에 큰 관심을 보이고 있습니다. 인재 한 명의 이직은 단순히 한 자리의 공백을 넘어 업무 연속성의 저하, 조직 분위기 악화, 신규 인력 채용 및 교육 비용 증가 등 여러 방면에 영향을 미칩니다. 특히 기술 중심 산업일수록 숙련 인력의 이탈은 회사의 경쟁력에 직접적인 타격이 됩니다.

이러한 배경에서 기업들은 사전에 이직 가능성이 높은 직원을 예측해 선제적으로 대응하려는 노력을 강화하고 있습니다. IBM에서 제공한 직원 이직 데이터 세트는 이러한 예측 모델을 개발하고 평가할 수 있는 좋은 실습 자료로, 다양한 인적 자원 관련 변수(예: 근무 기간, 직무 만족도, 부서 등)를 바탕으로 직원의 이직 여부를 분석할 수 있습니다.

이번 예제에서는 직원 이직 데이터를 기반으로 딥러닝 모델을 설계하고 학습해 인사 데이터에 숨겨진 패턴을 탐색하고 이직 가능성을 효과적으로 예측하는 방법을 익힙니다. 이는 단순한 기술적인 모델링을 넘어 데이터를 활용한 인사 의사결정의 좋은 실무 예시가 될 것입니다.

이번에 작성하는 코드 파일도 마찬가지로 별도의 파일을 생성하지 않고 '4일차_이진분류모델실습.ipynb' 파일에서 이어서 작성합니다. 기존 코드에서 데이터 전처리와 모델링 부분만을 변경하고 모델을 테스트할 예정입니다.

데이터 전처리

데이터 전처리에서는 먼저 데이터를 불러와 살펴본 다음, x, y 분할, 가변수화, 데이터 분할(train, val), 스케일링 그리고 데이터 로더를 준비합니다.

학습 데이터 소개

모델링에 사용할 데이터를 불러와 살펴보겠습니다.

CODE

```
path = "https://bit.ly/attri_csv"
data = pd.read_csv(path)
data.head()
```

OUTPUT

	Attrition	Age	Department	DistanceFromHome	Education	EmployeeNumber	Environment
0	0	33	Research & Development	7	3	817	
1	0	35	Research & Development	18	2	1412	
2	0	42	Research & Development	6	3	1911	
3	0	46	Sales	2	3	1204	
4	1	22	Research & Development	4	1	593	

데이터는 1,175명의 직원 정보를 담고 있으며 총 19개의 변수를 포함합니다. 이 중 목표값(target)은 'Attrition'으로 직원의 이직 여부를 나타냅니다. 값은 1(Yes)과 0(No)으로 이루어졌습니다. 입력값(feature)은 총 18개로서 이 중에는 모델링에서 사용하지 않을 정보도 있습니다. 예를 들어 사번(EmployeeNumber)은 단순히 직원을 구분하기 위한 식별번호일 뿐, 이직 여부를 설명하는 데 도움을 주지 않는 값입니다. 이런 값은 모델링에서 제외합니다. 비슷한 예로 주민번호, 사용자ID 등이 있습니다.

각 입력값(feature)의 내용과 설명은 [표 4-2]에서 보여 주고 있습니다.

[표 4-2] 직원 이직 여부 데이터 세트의 변수 설명

변수명	내용	부연 설명
Attrition	이직 여부	Yes=1, No=0
Age	나이	숫자
Department	현 부서	범주
DistanceFromHome	집-직장 거리	숫자(단위: 마일)
Education	교육 수준	범주(1→5: 숫자가 높을수록 고학력)
EmployeeNumber	사번	
EnvironmentSatisfaction	근무 환경 만족도	범주(1: 매우 낮음, 4: 매우 높음)
Gender	성별	범주
JobSatisfaction	직무 만족도	범주(1: 매우 낮음, 4: 매우 높음)
MaritalStatus	결혼 상태	범주(Single, Married, Divorced)
MonthlyIncome	월급	숫자
NumCompaniesWorked	근무 회사 수	숫자
OverTime	야근 여부	범주
PercentSalaryHike	급여 인상률(%)	숫자
StockOptionLevel	스톡옵션 수준	범주(0~3)
TotalWorkingYears	총 근무 연수	숫자
TrainingTimesLastYear	전년도 교육 횟수	숫자
WorkLifeBalance	일-삶 균형도	범주(1: 매우 낮음, 4: 매우 높음)
YearsAtCompany	현 직장 근무 기간	숫자

모델링을 제대로 하려면 입력변수의 의미를 파악하고 데이터 시각화와 통계 분석을 통해 비즈니스와 데이터를 충실히 파악하는 일이 선행되어야 합니다. 하지만 여기서는 이진 분류 모델링을 연습하는 것이 목적이므로 모델링 연습에 좀 더 초점을 두겠습니다.

x, y 구성

이제 목푫값(target)을 기준으로 x와 y를 저장합니다.

CODE

```
target = 'Attrition'
x = data.drop([target, 'EmployeeNumber'], axis = 1) ①
y = data.loc[:, target]
print(x.shape)
```

OUTPUT

```
(1175, 17)
```

① EmployeeNumber도 target 변수에 있는 Attrition과 함께 제외

x를 구성할 때는 target 변수뿐만 아니라 불필요한 변수도 확인하고 제외합니다. 코드 실행 결과 이 데이터는 1,175행 17열로 구성되어 있음을 알 수 있습니다.

가변수화

모든 범주형 변수를 리스트로 저장하고 가변수화를 수행합니다.

CODE

```
cat_cols = ['Department', 'Education', 'EnvironmentSatisfaction', 'Gender',
            'JobSatisfaction', 'MaritalStatus', 'OverTime', 'WorkLifeBalance']
x = pd.get_dummies(x, columns = cat_cols, drop_first=True)
x.shape
```

OUTPUT

```
(1175, 28)
```

범주형 변수가 많고 종류도 다양하기 때문에 모두 cat_cols라는 리스트에 담아 한꺼번에 가변수화를 수행합니다. 가변수화할 대상이 많으면 가변수화 이후 특징(feature) 수가 많이 증가합니다. 코드 실행 결과를 보면 열(feature) 수가 17에서 28로 증가했습니다.

데이터 분할 및 스케일링

데이터 분할과 스케일링을 수행합니다. 코드는 이전 예제들과 동일합니다.

CODE

```
# train, val 분할
x_train, x_val, y_train, y_val = train_test_split(x, y, test_size=.3)

# 스케일링
scaler = MinMaxScaler()
```

```
x_train = scaler.fit_transform(x_train)
x_val = scaler.transform(x_val)

# make_DataSet 입력을 위해 y_train, y_val을 넘파이 배열로 변환
y_train = y_train.values
y_val = y_val.values
```

데이터 로더 준비

전처리의 마지막으로 학습 데이터의 데이터 로더와 검증 데이터를 위한 텐서 데이터 세트를 준비합니다.

CODE

```
train_loader, x_val_ts, y_val_ts = make_DataSet(x_train, x_val, y_train, y_val, 32)
```

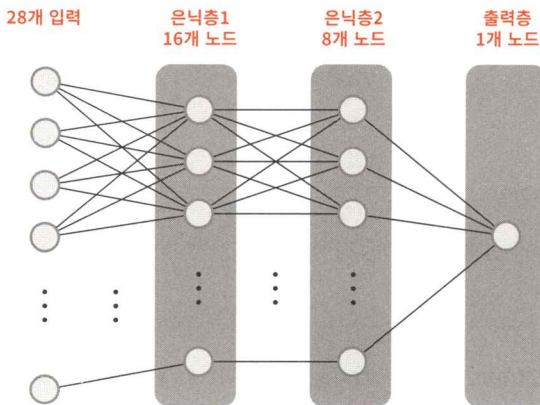
모델링을 수행할 데이터의 준비가 모두 끝났습니다. 모델링 단계로 넘어갑니다.

이진 분류 모델링

모델 구조 설계

모델은 [그림 4-13]과 같이 은닉층 2개와 출력층 1개, 총 3개의 층으로 구성합니다. 그림에서는 노드와 연결 일부만 표현하고 나머지는 생략했습니다.

데이터 전처리에서 생성한 28개의 입력 정보가 은닉층1에서는 16개의 노드와 연결되고, 은닉층2에서는 8개의 노드와 연결됩니다. 계속해서 은닉층2의 출력 정보가 1개의 출력층 노드와 연결되도록 모델 구조를 설계합니다.



[그림 4-13] 모델 구조 예시

이 모델 구조를 코드로 옮기면 다음과 같습니다.

CODE

```
from torchsummary import summary ①
n_feature = x.shape[1]
model = nn.Sequential(nn.Linear(n_feature, 16), nn.ReLU(), ②
                      nn.Linear(16, 8), nn.ReLU(), ③
                      nn.Linear(8, 1), nn.Sigmoid() ④
                      ),to(device)
summary(model, input_size=(n_feature,)) ⑤
```

OUTPUT

Layer (type)	Output Shape	Param #
Linear-1	[-1, 16]	464
ReLU-2	[-1, 16]	0
Linear-3	[-1, 8]	136
ReLU-4	[-1, 8]	0
Linear-5	[-1, 1]	9
Sigmoid-6	[-1, 1]	0
Total params: 609		
Trainable params: 609		
Non-trainable params: 0		
Input size (MB): 0.00		
Forward/backward pass size (MB): 0.00		
Params size (MB): 0.00		
Estimated Total Size (MB): 0.00		

- ① 지금까지 배우지 않았던 함수가 등장했습니다. 바로 모델 구조를 요약해 주는 summary 함수입니다. summary 함수는 torchsummary 라이브러리에 포함되어 있습니다.
- ② nn.Linear(n_feature, 16), nn.ReLU(): 첫 번째 은닉층에서는 입력으로 28개의 정보를 받아 16개의 노드와 연결되고 16개의 출력값을 계산합니다. 또한 은닉층이므로 활성화 함수 ReLU를 붙입니다.
- ③ nn.Linear(16, 8), nn.ReLU(): 두 번째 은닉층은 첫 번째 은닉층의 출력값(노드 수) 16개의 정보를 받아 8개의 노드로 출력값을 계산합니다. 역시 은닉층이므로 활성화 함수 ReLU를 붙입니다.
- ④ nn.Linear(8, 1), nn.Sigmoid(): 마지막으로 출력층은 두 번째 은닉층의 출력값 8개 정보와 연결되어 출력값 1개를 계산합니다. 이진 분류 모델이므로 최종 출력값을 시그모이드로 변환해 0~1 사이의 확률값으로 출력합니다.
- ⑤ summary 함수에는 두 가지 입력이 필요합니다. 첫 번째는 방금 만든 모델이고 두 번째는 input_size라는 값입니다. input_size는 모델에 들어가는 데이터가 어떤 구조(shape)인지 알려주는 값입니다. 지금 사용하고 있는 데이터는 하나의 표본이 여러 개의 특징(feature)으로 이루어진 1차원 구조이므로 n_feature에 feature의 수를 담아 지정합니다. 즉, 하나의 예측 단위 데이터가 몇 개의 변수로 이루어졌는지를 알려 주는 셈입니다.

요약 함수(torchsummary.summary)의 결과는 크게 3구역으로 나누어져 있습니다. 첫 번째 구역에서는 모델 구조를 요약하면서 각 층의 파라미터 수를 표시합니다. 두 번째 구역에서는 총 파라미터 수를 나타내고, 세 번째 구역에서는 MB 단위의 메모리 사용 용량을 표시합니다. 이 중에서 가장 많이 참고하는 것은 두 번째 구역에 있는 총 파라미터 수(Total params: 2,209)입니다. 이 숫자는 모델의 크기를 뜻합니다. 보통 모델이 크다 함은 파라미터 수가 많다는 말과 동일합니다. 거대 언어 모델(Large Language Model, LLM)의 ‘거대’는 파라미터 수의 규모가 매우 크다는 뜻입니다.

모델을 선언했으니 손실 함수와 옵티마이저도 준비합니다.

CODE

```
loss_fn = nn.BCELoss()
optimizer = Adam(model.parameters(), lr=0.001)
```

이번 코드에서는 학습률을 0.001로 지정했습니다. 학습률은 딥러닝 모델이 얼마나 빠르게 학습할지 결정하는 핵심 하이퍼파라미터 중 하나입니다. 학습률이 너무 크면 최적의 가중치를 찾지 못해 (필자가 학습 곡선을 설명할 때 표현한 것처럼) 들쭉날쭉할 수 있고, 반대로 너무 작으면 학습 속도가 지나치게 느려져 수백 번을 반복해도 제대로 수렴되지 않을 수 있습니다. 따라서 학습률은 적절한 값을 찾는 것이 중요합니다. 실전에서는 0.001과 같이 작은 학습률로 시작해 성능 변화를 보며 다양한 값을 시도합니다. 그중 모델의 성능이 가장 안정적으로 향상되는 지점의 학습률을 선택하는 것이 좋은 전략입니다.

학습

이제 에포크 수를 50으로 지정해 학습을 시킵니다. 에포크 역시 하이퍼파라미터이므로 실험으로 적절한 값을 찾아야 합니다.

CODE

```
epochs = 50
tr_loss_list, val_loss_list = [], []
for t in range(epochs):
    tr_loss = train(train_loader, model, loss_fn, optimizer, device)
    val_loss, _ = evaluate(x_val_ts, y_val_ts, model, loss_fn, device)
    tr_loss_list.append(tr_loss)
    val_loss_list.append(val_loss)
    print(f"Epoch {t+1}, train loss : {tr_loss:4f}, val loss : {val_loss:4f}")
```

OUTPUT

```
Epoch 1, train loss : 0.772043, val loss : 0.738960
Epoch 2, train loss : 0.694561, val loss : 0.635819
Epoch 3, train loss : 0.563159, val loss : 0.501232
...
Epoch 48, train loss : 0.309250, val loss : 0.343050
Epoch 49, train loss : 0.312502, val loss : 0.342936
Epoch 50, train loss : 0.308917, val loss : 0.343068
```

모델이 50번 반복 학습하면 학습 오차와 검증 오차는 점차 줄어듭니다.

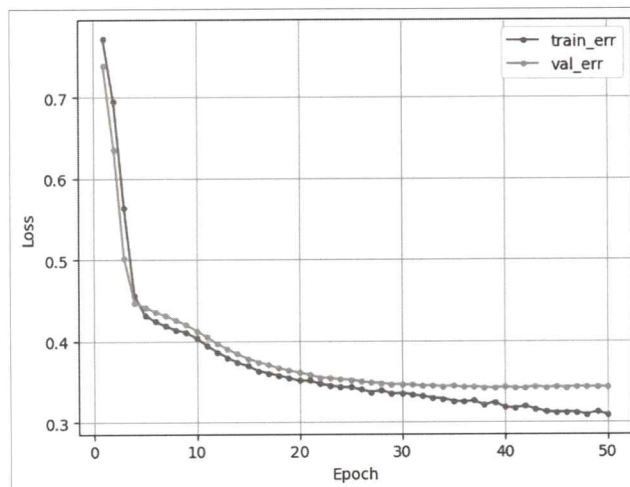
학습 곡선

이제 학습이 적절하게 진행되었는지 확인하기 위해 학습 곡선을 그립니다.

CODE

```
dl_learning_curve(tr_loss_list, val_loss_list)
```

OUTPUT



학습 곡선을 보면 학습 오차와 검증 오차는 4~5번째 에포크에서 꺾이고 이후부터는 점차 완만해집니다. 학습률과 에포크를 조절하는 방법은 6일차 ‘딥러닝 모델 성능 다루기’ 절에서 설명하겠습니다. 여기서는 성능 최적화보다는 이진 분류 모델링을 어떻게 구현하는지에 초점을 두기 바랍니다.

검증 평가

학습이 모두 끝났다면 모델의 성능을 검증 평가합니다.

예측

먼저 `evaluate` 함수로 예측합니다. 그리고 예측 결과(pred)를 넘파이 배열로 바꾸고 0.5 기준으로 반올림한 값, 0과 1을 저장합니다.

CODE

```
_, pred = evaluate(x_val_ts, y_val_ts, model, loss_fn, device)
pred = np.where(pred.numpy() > .5, 1, 0)
```

성능 평가

마지막으로 분류 결과 보고서인 `classification_report` 함수로 예측 결과를 평가합니다.

CODE

```
print(classification_report(y_val_ts.numpy(), pred))
```

OUTPUT

	precision	recall	f1-score	support
0.0	0.88	0.97	0.92	291
1.0	0.69	0.35	0.47	62
accuracy			0.86	353
macro avg	0.78	0.66	0.69	353
weighted avg	0.84	0.86	0.84	353

정분류율(Accuracy)이 0.86, 이직(1) 관점의 정밀도(Precision), 재현율(Recall), F1 Score가 각각 0.69, 0.35, 0.47로 나왔습니다.

모델에 검증 데이터를 입력해 예측한 결과, 전체적으로 이직 여부에 대한 예측 정확도는 0.86입니다. 이직이 예상된다고 예측한 직원 중 실제로 이직한 비율(정밀도, Precision)은 0.69이고, 실제로 이직한 직원 중 모델이 맞춘 비율(재현율, Recall)은 0.35입니다. 이 성능 평가의 결과값 역시 상대적인 값이므로 은닉층이 없는 간단한 모델(베이스라인 모델)을 먼저 만들어 성능을 측정하고 지금처럼 은닉층을 추가한 모델의 성능과 비교해 성능 개선이 있었는지 판단해야 됩니다.

정리

이번 차시에서는 먼저 은닉층에서 무슨 일이 벌어지는지 비즈니스 관점에서 설명했습니다. 은닉층의 역할은 특징을 재표현하는 단계로서 딥러닝 모델의 꽃이라고 할 수 있습니다.

이어서 딥러닝을 활용한 이진 분류 모델링을 단계별로 살펴보았습니다. 입력 데이터를 받아 은닉층에서 특징을 추출하고 출력층에서는 시그모이드(Sigmoid) 활성화 함수로 예측 확률을 계산하는 구조로 모델을 설계했습니다. 특히 모델이 예측한 확률을 바탕으로 임계값(보통 0.5)을 기준으로 0 또는 1을 판별하는 방식과 오차를 계산하기 위해 사용하는 손실 함수인 이진 교차 엔트로피(Binary Cross Entropy)의 원리도 함께 학습했습니다. 또한 파이토치에서는 목표값(target)을 정수형(0 또는 1)으로 만들어야 한다는 점도 꼭 기억하길 바랍니다.

3~4일 차에 걸쳐 회귀와 이진 분류 모델링을 배웠는데, 두 모델링의 주요 요점을 하나의 표로 정리했습니다. [표 4-3]처럼 회귀와 이진 분류에서 차이가 있는 지점은 출력층의 활성화 함수와 손실 함수입니다. 또한 이진 분류 모델은 학습하고 예측할 때 확률값을 0과 1로 나눠서(np.where) 평가한다는 점도 잊지 말길 바랍니다.

[표 4-3] 회귀, 이진 분류 설계 비교

구분	은닉층	출력층		최적화	
	활성화 함수	활성화 함수	노드 수	옵티마이저	손실 함수
회귀	ReLU	없음	1	Adam	nn.MSELoss
이진 분류	ReLU	Sigmoid	1	Adam	nn.BCELoss

이진 분류는 가장 기본적인 분류 문제이지만, 모델링의 전 과정을 직접 구성하면서 딥러닝의 핵심 원리를 이해할 수 있는 좋은 출발점입니다. 이후에 다룰 다중 분류나 더 복잡한 구조의 모델도 이진 분류에서 배운 내용을 바탕으로 합니다. 그리고 출력층의 구조와 손실 함수의 선택, 예측값의 해석 방식은 문제 유형에 따라 다르게 적용된다는 점도 꼭 기억해 두길 바랍니다.

다음 차시에서는 범주를 두 개가 아닌 세 개 이상으로 분류하는 모델인 다중 분류 모델링으로 한 걸음 더 나아가겠습니다.